ASSIGNMENT 9

Aayush Prasad **U24AI091**

1. To Write a C program to simulate producer-consumer problem using semaphores.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10 // Total items each thread will process

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for(int i = 0; i < NUM_ITEMS; i++) {
        int item = rand() % 100; // Generate a random item

        sem_wait(&empty); // Wait if buffer is full
        pthread_mutex_lock(&mutex); // Enter critical section

        buffer[in] = item;
        printf("Producer produced: %d at index %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // Leave critical section
        sem_post(&full); // Signal that buffer has new data

        sleep(1); // Simulate time taken to produce
    }
    return NULL;
}

void* consumer(void* arg) {
    for(int i = 0; i < NUM_ITEMS; i++) {
```

```c
        sem_wait(&full); // Wait if buffer is empty
        pthread_mutex_lock(&mutex); // Enter critical section

        int item = buffer[out];
        printf("Consumer consumed: %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // Leave critical section
        sem_post(&empty); // Signal that buffer has empty space

        sleep(2); // Simulate time taken to consume
    }
    return NULL;
}

int main() {
    pthread_t prod_thread, cons_thread;

    // Initialize semaphores and mutex

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);


    // Create threads

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    // Wait for threads to finish

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    // Cleanup

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Output:

```
:/mnt/c/Users/prati/Desktop/OS/assignment9$ gcc ques1.c -o ques1 -lpthread
:/mnt/c/Users/prati/Desktop/OS/assignment9$ ./ques1
```

```
Producer produced: 83 at index 0
Consumer consumed: 83 from index 0
Producer produced: 86 at index 1
Consumer consumed: 86 from index 1
Producer produced: 77 at index 2
Producer produced: 15 at index 3
Consumer consumed: 77 from index 2
Producer produced: 93 at index 4
Producer produced: 35 at index 0
Consumer consumed: 15 from index 3
Producer produced: 86 at index 1
Producer produced: 92 at index 2
Consumer consumed: 93 from index 4
Producer produced: 49 at index 3
Producer produced: 21 at index 4
Consumer consumed: 35 from index 0
Consumer consumed: 86 from index 1
Consumer consumed: 92 from index 2
Consumer consumed: 49 from index 3
Consumer consumed: 21 from index 4
```

2. Write a C program to simulate the concept of Dining-Philosophers problem.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    for(int i = 0; i < 3; i++) { // Each eats 3 timesa
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking

        // Deadlock avoidance: Last philosopher picks up right first
        if (id == NUM_PHILOSOPHERS - 1) {
            pthread_mutex_lock(&chopsticks[right])
            ;
            pthread_mutex_lock(&chopsticks[left]);
```

```
            pthread_mutex_lock(&chopsticks[left]);
            pthread_mutex_lock(&chopsticks[right]);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(2); // Eating

        // Put down chopsticks
        pthread_mutex_unlock(&chopsticks[left]);
        pthread_mutex_unlock(&chopsticks[right]);

        printf("Philosopher %d finished eating and put down chopsticks.\n",
id);
    }
    return NULL;
}

int main() {
    pthread_t philosophers[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    // Initialize mutexes for chopsticks
    for(int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
    }

    // Create philosopher threads
    for(int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    // Join threads
    for(int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }

    // Cleanup
    for(int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }

    return 0;

}
```

Output:

```
:/mnt/c/Users/prati/Desktop/OS/assignment9$  gcc ques2.c -o ques2 -lpthread
:/mnt/c/Users/prati/Desktop/OS/assignment9$ ./ques2
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 2 is thinking.
Philosopher 4 is eating.
Philosopher 1 is eating.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 0 is thinking.
Philosopher 4 finished eating and put down chopsticks.
Philosopher 4 is thinking.
Philosopher 1 finished eating and put down chopsticks.
Philosopher 1 is thinking.
Philosopher 3 is eating.
Philosopher 0 is eating.
Philosopher 0 finished eating and put down chopsticks.
Philosopher 0 is thinking.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 3 is thinking.
Philosopher 2 is eating.
Philosopher 4 is eating.
Philosopher 4 finished eating and put down chopsticks.
Philosopher 4 is thinking.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 2 is thinking.
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 1 finished eating and put down chopsticks.
Philosopher 1 is thinking.
Philosopher 3 finished eating and put down chopsticks.
Philosopher 3 is thinking.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 2 finished eating and put down chopsticks.
Philosopher 1 is eating.
```

3. Write a C program to simulate the concept of Readers-Writers problem.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t rw_mutex;       // Controls access to shared data
pthread_mutex_t mutex; // Protects the read_count variable
int read_count = 0;
int shared_data = 1;   // The resource being read/written

void* writer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 3; i++) {
        sem_wait(&rw_mutex); // Request exclusive access

        // Critical Section (Writing)
        shared_data = shared_data * 2;
        printf("Writer %d modified data to: %d\n", id, shared_data);

        sem_post(&rw_mutex); // Release access
        sleep(2);
    }
    return NULL;
}

void* reader(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 3; i++) {
        pthread_mutex_lock(&mutex); // Lock to update read_count
        read_count++;
        if (read_count == 1) {
            sem_wait(&rw_mutex); // First reader locks the resource from
writers
        }
        pthread_mutex_unlock(&mutex); // Unlock read_count

        // Critical Section (Reading)
        printf("Reader %d read data: %d (Active Readers: %d)\n", id,
shared_data, read_count);
        sleep(1);

        pthread_mutex_lock(&mutex); // Lock to update read_count
        read_count--;
        if (read_count == 0) {
            sem_post(&rw_mutex); // Last reader releases the resource
```

```
        }
        pthread_mutex_unlock(&mutex); // Unlock read_count

        sleep(1);
    }
    return NULL;
}
int main() {
    pthread_t r_threads[3], w_threads[2];
    int r_ids[3] = {1, 2, 3};
    int w_ids[2] = {1, 2};

    // Initialize semaphore and mutex
    sem_init(&rw_mutex, 0, 1);
    pthread_mutex_init(&mutex, NULL);

    // Create threads
    for(int i = 0; i < 3; i++) pthread_create(&r_threads[i], NULL, reader,
&r_ids[i]);
    for(int i = 0; i < 2; i++) pthread_create(&w_threads[i], NULL, writer,
&w_ids[i]);

    // Join threads
    for(int i = 0; i < 3; i++) pthread_join(r_threads[i], NULL);
    for(int i = 0; i < 2; i++) pthread_join(w_threads[i], NULL);

    // Cleanup
    sem_destroy(&rw_mutex);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

Output:

```
:/mnt/c/Users/prati/Desktop/OS/assignment9$ gcc ques3.c -o ques3 -lpthread
:/mnt/c/Users/prati/Desktop/OS/assignment9$ ./ques3
```

```
Reader 1 read data: 1 (Active Readers: 1)
Reader 2 read data: 1 (Active Readers: 3)
Reader 3 read data: 1 (Active Readers: 2)
Writer 1 modified data to: 2
Writer 2 modified data to: 4
Reader 1 read data: 4 (Active Readers: 1)
Reader 2 read data: 4 (Active Readers: 2)
Reader 3 read data: 4 (Active Readers: 3)
Writer 2 modified data to: 8
Writer 1 modified data to: 16
Reader 3 read data: 16 (Active Readers: 1)
Reader 1 read data: 16 (Active Readers: 2)
Reader 2 read data: 16 (Active Readers: 3)
Writer 2 modified data to: 32
Writer 1 modified data to: 64
```